

INF780 · VERIFICACIÓN Y VALIDACIÓN DE SOFTWARE

EXAMEN

PRIMER PARCIAL

Movies API · Implementación de endpoint + Testing integral

Gestión 2026 · Semestre I

COMPETENCIA EVALUADA

Diseña e implementa una nueva funcionalidad en una API REST construida con NestJS + TypeORM + PostgreSQL, aplicando criterios de verificación y validación de software mediante pruebas unitarias con mocks, pruebas de integración con HTTP y pruebas end-to-end contra una base de datos real, garantizando correctitud, robustez y mantenibilidad del software producido.

INSTRUCCIONES GENERALES

1. Trabaje individualmente. La copia implica la anulación del examen.
2. Parta del proyecto base INF780-MoviesApi entregado por el docente.
3. Clone el proyecto y trabaje allí.
4. Al terminar, entregue el enlace del repositorio y ejecute `npm run test` y `npm run test:e2e` frente al docente.

1. Contexto del problema

Usted trabaja en el equipo de backend de una plataforma que gestiona un catálogo de películas. La API actual expone operaciones CRUD básicas sobre el recurso `movies`, pero el equipo de frontend ha solicitado una nueva funcionalidad que permita buscar películas aplicando filtros combinables, para alimentar la pantalla principal del catálogo.

Su trabajo consiste en diseñar, implementar y verificar ese nuevo endpoint, garantizando su correcto funcionamiento mediante los tres niveles de pruebas estudiados durante la unidad: pruebas unitarias, pruebas de integración y pruebas end-to-end.

Endpoints existentes (como referencia)

POST	<code>/movies</code> Crear una nueva película
GET	<code>/movies</code> Obtener todas las películas
GET	<code>/movies/:id</code> Obtener una película por UUID
PATCH	<code>/movies/:id</code> Actualizar parcialmente una película
DELETE	<code>/movies/:id</code> Eliminar una película

2. Tarea a realizar

Agregue a la API un nuevo endpoint de búsqueda filtrada:

GET	<code>/movies/search</code> Buscar películas aplicando filtros opcionales
-----	---

Parámetros de consulta (query string)

Parámetro	Tipo	Requerido	Reglas de validación
<code>genre</code>	string	No	Debe ser un valor válido del enum Genre (action, comedy, drama, horror, sci-fi, thriller, romance, documentary, animation)
<code>year</code>	integer	No	Entero entre 1888 y 2030 (inclusive)
<code>minRating</code>	number	No	Decimal entre 0.0 y 10.0 (inclusive). Filtra películas con rating $\geq$ minRating

Comportamiento esperado

- Los tres parámetros son OPCIONALES y se combinan con operador lógico AND.
- Si no se envía ningún parámetro, el endpoint debe retornar todas las películas.
- Si algún parámetro es enviado pero no cumple las reglas de validación, se debe responder con HTTP 422 y un mensaje descriptivo.
- El endpoint siempre retorna un arreglo JSON (posiblemente vacío) con HTTP 200.
- El endpoint NO debe paginar; solo filtrar.
- Debe ser documentado con anotaciones `@ApiOperation`, `@ApiQuery` y `@ApiResponse` de Swagger.

Ejemplos de uso

```
# Sin filtros (retorna todas las películas)
GET /movies/search

# Solo por género
GET /movies/search?genre=sci-fi

# Por año y rating mínimo
GET /movies/search?year=2010&minRating=8.5

# Los tres filtros combinados
GET /movies/search?genre=drama&year=1994&minRating=9.0
```

IMPORTANTE · Orden de declaración de rutas

La ruta `/movies/search` debe declararse ANTES de la ruta `/movies/:id` en el controlador. Si no lo hace, NestJS interpretará "search" como un UUID y el endpoint nunca será alcanzado.

3. Entregables y distribución de puntaje

#	Sección	Archivo(s) a modificar / crear	Puntos
A	Implementación del endpoint	src/movies/dto/search-movies.dto.ts (nuevo) src/movies/movies.service.ts src/movies/movies.controller.ts	30
B	Pruebas unitarias del service	src/movies/movies.service.spec.ts	20
C	Pruebas de integración del controller	src/movies/movies.controller.spec.ts	20
D	Pruebas end-to-end	test/movies.e2e-spec.ts	20
E	Calidad de código y uso de Git	Historial de commits, tipado estricto, sin warnings del linter	10

4. Sección A · Implementación del endpoint (30 pts)

Paso 1. Crear el DTO de búsqueda

Cree el archivo `src/movies/dto/search-movies.dto.ts` con una clase `SearchMoviesDto` que declare las tres propiedades opcionales (`genre`, `year`, `minRating`) con sus respectivos decoradores de `class-validator` y `@ApiPropertyOptional` de `Swagger`.

Use `@Type(() => Number)` de `class-transformer` para convertir los query params (que siempre llegan como `string`) al tipo numérico correspondiente.

Paso 2. Agregar el método `search()` al service

En `MoviesService`, agregue el método `async search(filters: SearchMoviesDto): Promise<Movie[]>` que construya un objeto `where` dinámico y lo pase a `this.movieRepository.find({ where })`.

Para el filtro `minRating` debe usar el operador `MoreThanOrEqual` de `TypeORM`.

Si no se recibe ningún filtro, debe retornar todas las películas.

Paso 3. Exponer el endpoint en el controller

En `MoviesController`, agregue el método `@Get('search') search(@Query() filters: SearchMoviesDto)` que delegue al service.

Declárelo ANTES del método findOne (ruta /movies/:id) para evitar colisión de rutas. Anótelos con @ApiOperation, tres @ApiQuery (uno por parámetro) y @ApiResponse para 200 y 422.

Criterios de aceptación de la sección A

- El servidor arranca sin errores con npm run start:dev.
- Swagger (http://localhost:3000/api) muestra el nuevo endpoint con sus tres query params documentados.
- GET /movies/search?year=abc retorna HTTP 422.
- GET /movies/search?genre=unknown retorna HTTP 422.
- GET /movies/search (sin filtros) retorna HTTP 200 con un array.
- Todos los endpoints previos (POST, GET, PATCH, DELETE) siguen funcionando.

5. Sección B · Pruebas unitarias del service (20 pts)

Debe agregar al archivo existente `movies.service.spec.ts` los casos de prueba que verifiquen el comportamiento del método `search()`. Recuerde que el repositorio está mockeado (no hay base de datos), por lo que debe configurar el valor de retorno de `mockRepository.find` para cada escenario.

Casos mínimos requeridos

#	Descripción del caso	Verifica
B1	<code>search()</code> sin filtros debe llamar a <code>repository.find</code> con <code>where</code> vacío y retornar todas las películas	Lógica de filtros vacíos
B2	<code>search({ genre: 'drama' })</code> debe llamar a <code>repository.find</code> con <code>where: { genre: 'drama' }</code>	Construcción del filtro por género
B3	<code>search({ year: 2010 })</code> debe llamar a <code>repository.find</code> con <code>where: { year: 2010 }</code>	Construcción del filtro por año
B4	<code>search({ minRating: 8.0 })</code> debe llamar a <code>repository.find</code> con <code>where: { rating: MoreThanOrEqual(8.0) }</code>	Uso del operador <code>MoreThanOrEqual</code>
B5	<code>search({ genre, year, minRating })</code> debe combinar los tres filtros en el <code>where</code>	Combinación AND de filtros
B6	<code>search()</code> debe retornar un array vacío si el repositorio no encuentra coincidencias	Manejo de resultado vacío

Pista técnica

Para verificar que se usó `MoreThanOrEqual(8.0)`, importe el operador desde `typeorm` y compare en el `toHaveBeenCalledWith` usando `expect.objectContaining` y el valor del operador.

Alternativa aceptada: verificar la forma del argumento usando `expect.any(Object)` y complementar con un `toMatchObject`.

6. Sección C · Pruebas de integración del controller (20 pts)

Agregue al archivo `movies.controller.spec.ts` los casos de prueba que verifiquen el comportamiento del endpoint `GET /movies/search` a nivel HTTP. Use `supertest` sobre la

aplicación Nest ya configurada en el beforeEach (con ValidationPipe activo y errorHttpStatusCode: 422). El MoviesService está mockeado.

Casos mínimos requeridos

#	Petición HTTP	Código esperado	Verificación adicional
C1	GET /movies/search	200	Body es array; service.search llamado con {}
C2	GET /movies/search?genre=drama	200	service.search llamado con { genre: 'drama' }
C3	GET /movies/search?year=2010&minRating=8.5	200	service.search llamado con números (no strings)
C4	GET /movies/search?genre=invalid	422	Mensaje de error sobre enum Genre
C5	GET /movies/search?year=1500	422	Mensaje de error sobre rango de year
C6	GET /movies/search?minRating=11	422	Mensaje de error sobre rango de minRating

Importante

El caso C3 debe verificar explícitamente que los query params lleguen al service como number, no como string. Esto valida que la conversión con `@Type(() => Number)` esté funcionando correctamente.

7. Sección D · Pruebas end-to-end (20 pts)

Agregue al archivo `test/movies.e2e-spec.ts` un bloque describe dedicado al endpoint `/movies/search`. A diferencia de las pruebas anteriores, estas corren contra PostgreSQL real (base `movies_api_test`) y la aplicación completa.

Dataset a sembrar en `beforeAll` del describe E2E

```
const seedMovies = [  
  { title: 'Inception',      director: 'C. Nolan',  genre: 'sci-fi',  year: 2010,  
    rating: 8.8 },  
  { title: 'Interstellar',   director: 'C. Nolan',  genre: 'sci-fi',  year: 2014,  
    rating: 8.6 },  
  { title: 'The Godfather',  director: 'F. Coppola', genre: 'drama',   year: 1972,  
    rating: 9.2 },  
  { title: 'Pulp Fiction',   director: 'Q. Tarantino', genre: 'drama',   year: 1994,  
    rating: 8.9 },  
  { title: 'The Dark Knight', director: 'C. Nolan',  genre: 'action',  year: 2008,  
    rating: 9.0 },  
  { title: 'Toy Story',      director: 'J. Lasseter', genre: 'animation', year: 1995,  
    rating: 8.3 },  
];
```

Casos mínimos requeridos

#	Petición HTTP	Resultado esperado
D1	GET /movies/search	6 películas (todas las sembradas)
D2	GET /movies/search?genre=sci-fi	2 películas (Inception, Interstellar)
D3	GET /movies/search?year=1994	1 película (Pulp Fiction)
D4	GET /movies/search?minRating=9.0	2 películas (The Godfather, The Dark Knight)
D5	GET /movies/search?genre=drama&minRating=9.0	1 película (The Godfather)
D6	GET /movies/search?genre=horror	0 películas (array vacío, HTTP 200)
D7	GET /movies/search?year=2030	0 películas (array vacío, HTTP 200)
D8	GET /movies/search?year=invalid	HTTP 422 con mensaje de validación

Recuerde

- Limpie la tabla `movies` en `beforeAll` antes de insertar el dataset.

- No dependa del orden en que PostgreSQL retorna los registros: verifique cantidad y contenido por título.
- Use `await dataSource.query('DELETE FROM movies');` para la limpieza final en `afterAll`.

8. Sección E · Calidad y Git (10 pts)

Criterio	Puntos
Proyecto clonado y en funcionamiento	2
Mínimo 4 commits descriptivos distribuidos (no todo en un solo commit final)	3
<code>npm run lint</code> sin errores ni warnings	2
Tipado estricto: sin uso de <code>any</code> fuera del tipo <code>MockRepository</code> ya existente	2
Código formateado con <code>npm run format</code>	1

Entrega final

Envíe al docente el enlace del repositorio (<https://github.com/<usuario>/INF780-parcial1-<apellido>>) y ejecute los comandos `npm run test` y `npm run test:e2e` frente a él. Ambos deben terminar en verde.

— Fin del examen —